

EPIC 09- Task 05: Final System Design Review

Production Readiness Review — Rapido/Bike Taxi Microservices System

Area	Review / Implementation
1. Architecture	Use API Gateway with separate User, Driver, Ride, Payment, Location and Notification microservices.
2. Microservice Boundaries	Each service has a specific business function and should have its own database/schema. Avoid sharing business logic between services.
3. API Design	Use REST APIs with proper HTTP methods, status codes, validation, versioning (/api/v1/...) and Swagger/OpenAPI documentation.
4. Database Design	Use PostgreSQL/MySQL for persistent data. Maintain separate schemas/databases for services where appropriate. Use indexes, constraints, and transactions.
5. Authentication	Implement Spring Security with JWT-based authentication. Validate the token at the API Gateway/services.
6. Authorization	Implement role-based access control such as USER, DRIVER, and ADMIN. Users can access only authorized resources.
7. Service Communication	Use REST for synchronous communication and Kafka for asynchronous events such as ride status, payment, and notifications.

8. Kafka	Define topics such as ride events, payment events, and notification events. Use consumer groups and handle failed messages safely.
9. Redis	Use Redis for caching, driver availability/location data and frequently accessed information. Configure TTL where appropriate.
10. Circuit Breaker	Use Resilience4j circuit breakers, timeouts and fallback mechanisms so one unavailable service does not bring down the entire system.
11. Error Handling	Use centralized exception handling with @ControllerAdvice. Return consistent error responses containing status, message, and timestamp.
12. Logging	Use structured logs with correlation/request IDs. Do not log passwords, JWTs, payment details, or other sensitive information.
13. Monitoring	Use Spring Boot Actuator for health checks and metrics. Monitor CPU, memory, request latency, error rates, and service availability.
14. Testing	Implement unit, integration, API, service-communication and failure tests. Test both successful and failure scenarios.
15. Docker	Create Dockerfiles for all services and a docker-compose.yml for Gateway, microservices, database, Redis and Kafka.
16. Scalability	Keep services stateless where possible and allow independent horizontal scaling. Use Redis/Kafka to support distributed workloads.

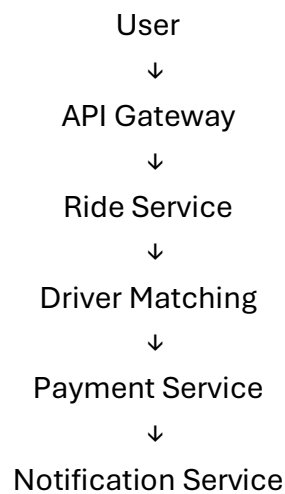
17. High Availability

Use multiple service instances, health checks, retries/timeouts, and redundant infrastructure to avoid single points of failure.

18. Security

Use HTTPS, secure secrets/configuration, input validation, least-privilege access and protection against common API vulnerabilities.

Final Business Workflow



Final Verdict

Production readiness: READY after validation of all checklist items.

Before production deployment, verify that **security, testing, monitoring, logging, database reliability, Kafka/Redis availability, Docker deployment, scalability and failure handling** have all been tested successfully.